



Angular Avançado

Sessão 6

Navegação com o Router

Onde ficámos na Sessão 5

Boarding List — o que o exercício consolidou

Estado com signals

- `signal<Passenger[]>` como fonte de verdade
- `update` devolve **array novo** e **objeto novo**
- `push()` e `p.boarded = ...` = mutação: nada atualiza

computed encadeados

- `total · boardedCount · allBoarded`
- `visible` filtra por `filter()`
- Só recalculam quando o que leem muda

Template e componentes

- `@for ... track + @empty`
- `@let f = filter() + @switch + @default never`
- `PassengerRow`: input / output → model
- `[(boarded)]="p.boarded"` compila, mas **muta**

Hoje: o Router — a Skyboard passa a ter páginas, parâmetros, sub-navegação e lazy loading.

Agenda de hoje

01 **Primeiros passos** Rotas · router-outlet · routerLink · navigate() ≈ 30 min

02 **Parâmetros** ActivatedRoute · matrix · input binding ≈ 25 min

03 **Child routes** Outlets dentro de outlets · links relativos ≈ 25 min

04 **Lazy loading** Rotas de um domínio a pedido: loadChildren ≈ 15 min

Na Sessão 7: loadComponent, preloading, query string e path vs hash.



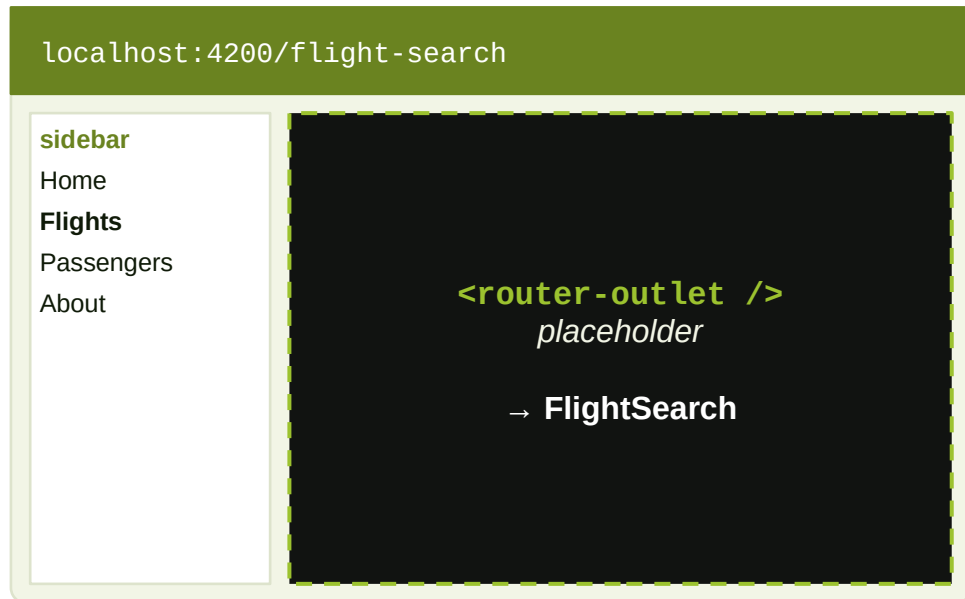
Módulo 3 · Routing

Router: primeiros passos

Várias páginas numa single-page application — com o URL a acompanhar

Porquê um Router?

Mostrar/esconder componentes não chega — o URL tem de acompanhar



O que o URL nos dá

- Botão **back** e histórico do browser
- **Bookmarks** e links partilháveis
- **Refresh** sem perder a página
- O path aponta para uma entrada de configuração → o Router **ativa** o componente

`app.routes.ts` tem as rotas disponíveis desde o arranque. As features podem trazer as suas, carregadas a pedido — lazy loading, ainda hoje.

O que vamos navegar

Três componentes novos + os dois que já existem

```
terminal
```

```
ng g c shell/home
```

```
ng g c shell/about
```

```
ng g c domains/ticketing/feature-booking/passenger-search
```

Rota	Componente	Pasta	Estado
/home	Home	shell/home	novo — boas-vindas + botão
/flight-search	FlightSearch	domains/ticketing/feature-booking	já existe
/passenger-search	PassengerSearch	domains/ticketing/feature-booking	novo — placeholder
/boarding	BoardingList	domains/ticketing/feature-boarding	do exercício de hoje
/about	About	shell/about	novo — placeholder

Configurar as rotas: app.routes.ts

```
src/app/app.routes.ts
import { Routes } from '@angular/router';
// + imports de Home, FlightSearch, PassengerSearch, BoardingList,
About

export const routes: Routes = [
  { path: '', pathMatch: 'full', redirectTo: 'home' }, // default
  { path: 'home', component: Home },
  { path: 'flight-search', component: FlightSearch },
  { path: 'passenger-search', component: PassengerSearch },
  { path: 'boarding', component: BoardingList },
  { path: 'about', component: About },
  { path: '**', redirectTo: 'home' }, // catch-
all
];
```

Anatomia de uma rota

- path: segmento do URL, **sem** / inicial
- component: o que o Router ativa
- redirectTo: troca o URL e volta a procurar
- '' = rota por omissão (localhost:4200)
- ** = qualquer coisa que não bateu antes

pathMatch e ordem: a primeira que casa ganha

URL pedido	Rota que casa	Resultado
localhost:4200	" (pathMatch: 'full')	redirect → /home
/flight-search	'flight-search'	FlightSearch
/boarding	'boarding'	BoardingList
/xpto	'**'	redirect → /home

Pergunta: o que acontece se o ** for a primeira entrada do array?

Resposta: todas as navegações vão parar a /home — o catch-all casa sempre primeiro.

Porquê pathMatch: 'full'?

- Por omissão, match por **prefixo**
- '' é prefixo de **qualquer** string
- Sem full, a rota por omissão apanhava tudo

Ordem importa

- Avaliadas de cima para baixo
- Específicas antes de genéricas; ** sempre em último

Registrar e reservar o lugar: provideRouter + router-outlet

```
app.config.ts / app.ts
// app.config.ts – já feito pelo CLI
providers: [
  provideBrowserGlobalErrorListeners(),
  provideRouter(routes),
]

// app.ts
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  imports: [RouterOutlet, Navbar, Sidebar], // sem
  FlightSearch
  templateUrl: './app.html',
})
```

```
app.html
<div class="wrapper">
  <div class="sidebar">
    <app-sidebar />
  </div>
  <div class="main-panel">
    <app-navbar />
    <div class="content">
      <!-- <app-flight-search /> -->
      <router-outlet />
    </div>
  </div>
</div>
```

O componente deixa de estar no template: é o Router que o ativa dentro do `<router-outlet />`.

Links que ativam rotas: routerLink + routerLinkActive

```
shell/sidebar/sidebar.ts
import { RouterLink, RouterLinkActive } from
 '@angular/router';

@Component({
  selector: 'app-sidebar',
  imports: [RouterLink, RouterLinkActive],
  templateUrl: './sidebar.html',
})
export class Sidebar {}
```

```
shell/sidebar/sidebar.html
<ul class="nav">
  <li routerLinkActive="active">
    <a routerLink="home"><p>Home</p></a>
  </li>
  <li routerLinkActive="active">
    <a routerLink="flight-search"><p>Flights</p></a>
  </li>
  <!-- passenger-search · boarding · about -->
</ul>
```

routerLink

- Valor = o path da rota; navega **sem** recarregar
- href="/home" recarrega a app inteira — evitar

routerLinkActive

- Põe a classe quando o elemento **ou um descendente** está ativo → vai no
- .active já existe no styles.css

Alternativa: isActive() devolve um Signal

Angular 21.1+ · quando o estado da rota tem de entrar em lógica

```
sidebar – variante com isActive
import { computed, inject } from '@angular/core';
import { isActive, Router, RouterLink } from '@angular/router';

export class Sidebar {
  private readonly router = inject(Router);

  protected readonly searchActive = isActive('/flight-search', this.router);
  protected readonly boardingActive = isActive('/boarding', this.router);

  // é um Signal<boolean> → entra no grafo como outro qualquer
  protected readonly inTicketing = computed(
    () => this.searchActive() || this.boardingActive();
  )
}

<li [class.active]="searchActive()"> ... </li>
```

Quando usar

- Só CSS → routerLinkActive (fica no Skyboard)
- Lógica (computed, effect, @if) → isActive
- 3.º arg.: { paths: 'exact' } para match exato (por omissão: subset)

Experimentar: o que deve acontecer

- | | | |
|----------|----------------------------------|---|
| 1 | Clicar nos links | O URL muda; na Network (filtro Doc) não há novo pedido do documento |
| 2 | Back / forward | O browser navega entre as páginas visitadas |
| 3 | Refresh em /flight-search | Continua na pesquisa — o dev server devolve o index.html |
| 4 | localhost:4200 · /xpto | Ambos acabam em /home (default e catch-all) |
| 5 | Sair e voltar a Flights | A pesquisa recomeça do zero — porquê? |

5 → o componente é **destruído** ao sair e **recriado** ao voltar: o estado dele (signals) vai junto. Guardar estado entre páginas = serviços (módulo 7).

Navegar a partir do código: Router.navigate()

```
shell/home/home.ts / .html
import { Component, inject } from '@angular/core';
import { Router } from '@angular/router';

export class Home {
  private readonly router = inject(Router);

  protected startSearch(): void {
    this.router.navigate(['/flight-search']);
  }
}

<!-- home.html -->
<button type="button" (click)="startSearch()">Start searching</button>
```

Segmentos

- Cada entrada do array = 1 segmento
- `['/a', 'b', id]` com `id = 17` → `/a/b/17`

A seguir

É assim que se chega a rotas com parâmetros: `/flight-edit/17`.

Link no template → `routerLink`. Decisão no código (depois de guardar, de validar, de um clique com lógica) → `router.navigate()`.

Três formas de passar parâmetros no URL

Tipo	Exemplo	Pertence a	Na config?
Segmento	flight-edit/17	posição no path	sim — :id
Matrix	flight-edit/17;showDetails=true	o segmento onde está	não
Query string	flight-edit/17?expertMode=true	o URL inteiro	não

```
/ticketing/booking/flight-edit/17;showDetails=true/passengers;orderBy=name  
// showDetails → segmento flight-edit/17 · orderBy → segmento passengers
```

Para parâmetros **com nome**, o Router usa **matrix** por omissão: cada parâmetro fica agarrado ao seu segmento — e, com child routes, ao seu componente.

Ler parâmetros: ActivatedRoute

O mecanismo — um Observable com os parâmetros da rota ativa

```
feature-booking/flight-edit/flight-edit.ts
import { Component, inject, signal } from '@angular/core';
import { ActivatedRoute } from '@angular/router';

export class FlightEdit {
  private readonly route = inject(ActivatedRoute);
  protected readonly id = signal(0);
  protected readonly showDetails = signal(false);

  constructor() {
    // segmento (:id) + matrix (;showDetails=true)
    this.route.paramMap.subscribe((paramMap) => {
      this.id.set(Number(paramMap.get('id') ?? 0));
      this.showDetails.set(paramMap.get('showDetails') === 'true');
    });
  }
}
```

O que reparar

- paramMap é um **Observable**: volta a emitir se o componente for **reutilizado**
- snapshot só tem os valores da criação
- No URL é tudo **string** → conversão à mão
- Sem unsubscribe: completa quando o componente é destruído

Configurar a rota e ligar a partir da pesquisa

```
// app.routes.ts – só o parâmetro de segmento se declara (antes do '**')
{ path: 'flight-edit/:id', component: FlightEdit },
```

```
o pai: cada entrada = 1 segmento; objeto = matrix
<!-- flight-search.html (+ RouterLink nos imports) -->
<app-flight-card [item]="flight" ...>
  <button class="actions"
    [routerLink]="['../flight-edit', flight.id,
      { showDetails: true }]">
    Edit
  </button>
  ...
</app-flight-card>
```

```
o card: não sabe nada de routing
<!-- flight-card.html: slot próprio para ações -->
<div class="btn-row">
  @if (!selected()) {
    <button (click)="select()">Select</button>
  } @else { ... Remove ... }
  <ng-content select=".actions" />
</div>
<div class="card-extra"><ng-content>...</ng-content></div>
<h5><ng-content select=".footer" /></h5>
```

Voo #3 → /flight-edit/3;showDetails=true · ../ porque flight-edit é **irmão** de flight-search

Refactor: parâmetros como inputs

withComponentInputBinding() — a versão que fica no Skyboard

```
// app.config.ts — ativar ANTES de mudar o componente
provideRouter(routes, withComponentInputBinding()),
```

flight-edit.ts — o template não muda

```
import { booleanAttribute, Component, effect, input,
        numberAttribute } from '@angular/core';

export class FlightEdit {
  // :id (segmento)
  readonly id = input.required({ transform: numberAttribute });
  // ;showDetails (matrix)
  readonly showDetails = input(false, { transform: booleanAttribute });

  constructor() {
    effect(() =>
      console.log('id', this.id(), 'showDetails', this.showDetails()));
  }
}
```

vs ActivatedRoute

- Sem inject, sem subscribe
- Mesmo **nome** do parâmetro → mesmo input
- transform em vez de conversão à mão
- Em falta → undefined → valor por omissão

Child routes: outlets dentro de outlets

localhost:4200/booking/flight-search

App

router-outlet → BookingNavigation

Flights | Passengers

router-outlet (filho)
→ FlightSearch

Como se lê

- booking ativa BookingNavigation no outlet da App
- flight-search ativa FlightSearch no outlet do BookingNavigation
- Cada nível do URL ↔ um nível de outlet
- Menu secundário + outlet = áreas com sub-navegação

O componente com o outlet filho: BookingNavigation

```
ng g c domains/ticketing/feature-booking/booking-navigation --flat
```

```
feature-booking/booking-navigation.ts
import { RouterLink, RouterLinkActive, RouterOutlet } from
 '@angular/router';

@Component({
  selector: 'app-booking-navigation',
  imports: [RouterLink, RouterLinkActive, RouterOutlet],
  templateUrl: './booking-navigation.html',
})
export class BookingNavigation {}
```

```
feature-booking/booking-navigation.html
<div class="card"><div class="card-body">
  <ul class="nav nav-secondary">
    <li><a routerLink="/flight-search"
      routerLinkActive="active">Flights</a></li>
    <li><a routerLink="/passenger-search"
      routerLinkActive="active">Passengers</a></li>
  </ul>
</div></div>
<div class="mt-40"><router-outlet /></div>
```

--flat: sem pasta própria — fica na raiz de feature-booking/, como porta de entrada da feature. As classes nav-secondary, card e mt-40 já existem no styles.css.

Configurar as child routes

app.routes.ts – substitui flight-search, flight-edit e passenger-search

```
{
  path: 'booking',
  component: BookingNavigation,
  children: [
    { path: '', pathMatch: 'full', redirectTo: 'flight-search' },
    { path: 'flight-search', component: FlightSearch },
    { path: 'flight-edit/:id', component: FlightEdit },
    { path: 'passenger-search', component: PassengerSearch },
  ],
},
{ path: 'boarding', component: BoardingList }, // fica na raiz, por agora
```

Mudanças

- '' em children = omissão **dentro** de booking
- Sidebar: Flights + Passengers → **Booking** (routerLink="booking")
- Home:
navigate([' /booking/flight-search'])
- O link Edit (./flight-edit) continua a funcionar: são irmãos na mesma

Esqueceram-se de atualizar a Home? O ** apanha /flight-search e manda para /home — sem erro. URLs em strings partem em silêncio.

Links relativos: contar segmentos, não rotas

Estou em	routerLink	Vou para
/booking	./flight-search (ou flight-search)	/booking/flight-search
/booking/flight-search	['../flight-edit', 3]	/booking/flight-edit/3
/booking/flight-edit/3	../flight-search	/booking/flight-edit/flight-search X (Id: NaN)
/booking/flight-edit/3	../../flight-search	/booking/flight-search ✓
qualquer sítio	/booking/flight-search	absoluto: sempre o mesmo

.. sobe **um segmento** do URL. `flight-edit/:id` ocupa **dois** — daí o `../../` no botão "Back to search" do `FlightEdit`.

Relativo sobrevive a mudanças de prefixo (ex.: `/ticketing/...`); absoluto é mais fácil de ler, mas parte quando o prefixo muda.

Lazy loading: rotas de um domínio a pedido

```
// domains/ticketing/ticketing.routes.ts
const ticketingRoutes: Routes = [
  { path: 'booking', component: BookingNavigation,
    children: [ /* ' ', flight-search, flight-edit/:id, passenger-search */ ] },
  { path: 'boarding', component: BoardingList },
];
export default ticketingRoutes;
```

```
// app.routes.ts – já não importa nada de ticketing
{
  path: 'ticketing',
  loadChildren: () => import('./domains/ticketing/ticketing.routes'),
},
```

Sem export default: import('...').then((m) => m.ticketingRoutes)

Consequências

- Prefixo novo:
/ticketing/booking/...,
/ticketing/boarding
- Atualizar sidebar e Home
- Links relativos: intactos

Resumo da sessão

Rotas e links

- Routes: path → component
- '+' + pathMatch: 'full' . ** em último
- routerLink · routerLinkActive · isActive()
- router.navigate([...])

Parâmetros e hierarquia

- Segmento :id · matrix ;x=1
- ActivatedRoute → withRouterInputBinding()
- children + outlet filho
- .. sobe um **segmento**

Lazy loading

- loadChildren + import() dinâmico
- export default nas rotas do domínio
- Prefixo novo: absolutos partem, relativos não
- Sair da rota destrói o componente

Sessão 7: loadComponent, preloading, query string e path vs hash — depois, serviços e Dependency Injection. Antes: git pull → tag sessao-06.